



动态代理实验

一、环境配置

1、pom依赖的引入

二、jdk动态代理实验

1、编写 UserDao

2、编写 UserDaoImpl

3、编写 MyAspect

4、编写 MyProxy

5、编写 JDKTest

三、cglib动态代理实验

1、编写 UserDao

2、编写 MyAspect

3、编写 CglibProxy

4、编写 CglibTest

一、环境配置

1、pom依赖的引入

```
<dependencies>
  <dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-webmvc</artifactId>
    <version>6.1.4</version>
  </dependency>
  <dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-test</artifactId>
    <version>6.1.4</version>
    <scope>test</scope>
  </dependency>
  <dependency>
    <groupId>org.aspectj</groupId>
```

```
        <artifactId>aspectjweaver</artifactId>
        <version>1.9.4</version>
    </dependency>
    <dependency>
        <groupId>junit</groupId>
        <artifactId>junit</artifactId>
        <version>4.13</version>
        <scope>compile</scope>
    </dependency>
</dependencies>
```

二、jdk动态代理实验

1、编写UserDao

在项目的src/main/java目录下，创建一个com.itheima.demo01包，在该包下创建接口 UserDao，在 UserDao 接口中编写添加和删除的方法。

```
public interface UserDao {
    public void insert();
    public void update();
    public void delete();
}
```

2、编写UserDaoImpl

创建 UserDao 接口的实现类 UserDaoImpl，分别实现接口中的方法。

```
public class UserDaoImpl implements UserDao {
    public void insert() {
        System.out.println("插入");
    }

    public void update() {
        System.out.println("更新");
    }
}
```

```

        public void delete() {
            System.out.println("删除");
        }
    }
}

```

3、编写MyAspect

创建切面类MyAspect，在该类中定义一个模拟权限检查的方法和一个模拟日志记录的方法，这两个方法就是切面中的通知。

```

public class MyAspect {    // 切面类：存在多个通知Advice（增强的方法）
    public void before() {
        System.out.println("目标对象调用之前打印");
    }
    public void after() {
        System.out.println("目标对象调用之后打印");
    }
}

```

4、编写MyProxy

创建代理类MyProxy，该类需要实现InvocationHandler接口设置代理类的调用处理程序。在代理类中，通过newProxyInstance()生成代理方法。

```

public class MyProxy implements InvocationHandler {
    private UserDao userDao;

    public Object createProxy(UserDao userDao) {
        this.userDao = userDao;
        ClassLoader classLoader = MyProxy.class.getClassLoader(); // 1.类加载器
        Class[] classes = userDao.getClass().getInterfaces(); // 2.被代理对象实现的所有接口
        return Proxy.newProxyInstance(classLoader, classes, this); // 3.返回代理对象
    }
}

```

```

@Override
public Object invoke(Object proxy, Method method, Object[] args) throws Throwable {
    // 创建切面对象
    MyAspect myAspect = new MyAspect();

    // 检查权限
    myAspect.before();

    // 在目标类上调用方法
    Object obj = method.invoke(userDao, args);

    // 记录日志
    myAspect.after();
    return obj;
}
}

```

5、编写JDKTest

创建测试类JDKTest。在该类中的main()方法中创建代理对象jdkProxy和目标对象userDao，然后从代理对象jdkProxy中获得对目标对象userDao增强后的对象userDao1，最后调用userDao1对象中的添加和删除方法。

```

public class JDKTest {
    public static void main(String[] args) {
        // 创建代理对象
        MyProxy jdkProxy = new MyProxy();

        // 创建目标对象
        UserDao userDao = new UserDaoImpl();

        // 从代理对象中获取增强后的目标对象
        UserDao userDao1 = (UserDao) jdkProxy.createProxy(userDao);

        // 执行方法
    }
}

```

```
        userDao1.insert();
        userDao1.update();
        userDao1.delete();
    }
}
```



JDK动态代理存在缺陷，它只能为接口创建代理对象，当需要为类创建代理对象时，就需要使用CGLib（Code Generation Library）动态代理，CGLib动态代理不要求目标类实现接口，它采用底层的字节码技术，通过继承的方式动态创建代理对象。Spring的核心包已经集成了CGLib所需要的包，所以开发中不需要另外导入JAR包。

三、cglib动态代理实验

1、编写UserDao

在项目的src/main/java目录下创建一个com.itheima.demo02包，在该包下创建目标类UserDao，在该类中编写添加用户和删除用户的方法。

```
public class UserDao {
    public void insert() {
        System.out.println("插入");
    }

    public void update() {
        System.out.println("更新");
    }

    public void delete() {
        System.out.println("删除");
    }
}
```

2、编写MyAspect

创建切面类MyAspect，在该类中定义一个模拟权限检查的方法和一个模拟日志记录的方法，这两个方法就是切面中的通知。

```
public class MyAspect {    // 切面类：存在多个通知Advice（增强的方法）
    public void before() {
        System.out.println("目标对象调用之前打印");
    }
    public void after() {
        System.out.println("目标对象调用之后打印");
    }
}
```

3、编写CglibProxy

创建代理类CglibProxy，该代理类需要实现MethodInterceptor接口用于设置代理类的调用处理程序，并实现接口中的intercept()方法

```
public class CglibProxy implements MethodInterceptor {
    // 代理方法
    public Object createProxy(Object target) {
        // 创建一个动态类对象
        Enhancer enhancer = new Enhancer();
        // 确定需要增强的类，设置其父类
        enhancer.setSuperclass(target.getClass());
        // 添加回调函数
        enhancer.setCallback(this);
        // 返回创建的代理类
        return enhancer.create();
    }

    @Override
    public Object intercept(Object proxy, Method method, Object[] args, MethodProxy methodProxy) throws Throwable {
        // 创建切面对象
        MyAspect myAspect = new MyAspect();

        // 检查权限
    }
}
```

```

        myAspect.before();

        // 在目标类上调用方法
        Object obj = methodProxy.invokeSuper(proxy, args);

        // 记录日志
        myAspect.after();
        return obj;
    }
}

```

4、编写CglibTest

创建测试类CglibTest，在main()方法中首先创建代理对象cglibProxy和目标对象userDao，然后从代理对象cglibProxy中获得增强后的目标对象userDao1，最后调用userDao1对象的添加和删除方法。

```

public class CglibTest {
    public static void main(String[] args) {
        // 创建代理对象
        CglibProxy cglibProxy = new CglibProxy();

        // 创建目标对象
        UserDao userDao = new UserDao();

        // 获取增强后的目标对象
        UserDao userDao1 = (UserDao) cglibProxy.createProxy(
            userDao);

        // 执行方法
        userDao1.insert();
        userDao1.update();
        userDao1.delete();
    }
}

```